

```

1  #include <stdio.h>
2
3  #define MAX_N 100
4
5  typedef struct {
6      int n;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 Graph init_graph(int n) {
11     Graph G;
12     G.n = n;
13     for (int i = 0; i <= n; i++) {
14         for (int j = 0; j <= n; j++) {
15             G.A[i][j] = 0;
16         }
17     }
18
19     return G;
20 }
21
22 void add_edge(Graph *G, int u, int v) {
23     if (u < 1 || u > G->n || v < 1 || v > G->n || u == v) return;
24
25     G->A[u][v] = 1;
26     G->A[v][u] = 1;
27 }
28
29 void neighbours(Graph G, int u, int adjList[]) {
30     int k = 0;
31     for (int v = 1; v <= G.n; v++) {
32         if (G.A[u][v] != 0) {
33             adjList[k] = v;
34             k++;
35         }
36     }
37     adjList[k] = 0;
38 }
39
40
41 int DFS(Graph G, int u, int before, int mark[]) {
42     mark[u] = 1;
43     int list[G.n + 1], v, i = 0;
44     neighbours(G, u, list);
45
46     while ((v = list[i]) != 0) {
47         if (!mark[v]) {
48             if (DFS(G, v, u, mark)) return 1;
49         }
50         else if (v != before) return 1;
51
52         i++;
53     }
54
55     return 0;
56 }

```

```

58 □ int main() {
59   int n, m, u, v, e;
60   scanf("%d%d", &n, &m);
61   Graph G = init_graph(n);
62
63 □   for (e = 0; e < m; e++) {
64       scanf("%d%d", &u, &v);
65       add_edge(&G, u, v);
66   }
67
68   int flag = 0, mark[G.n + 1];
69 □   for (int i = 0; i <= G.n; i++) {
70       mark[i] = 0;
71   }
72 □   for (int i = 1; i <= G.n && !flag; i++) {
73 □       if (!mark[i]) {
74           flag = DFS(G, v, 0, mark);
75       }
76   }
77
78   if (flag) puts("YES");
79   else puts("NO");
80
81   return 0;
82 }

```